



Fundamentals of Software Engineering

LECTURE: AGILE AND EXTREME PROGRAMMING

Project Failure – the trigger for Agility

- ▶ One of the primary causes of project failure was the **extended period of time it took to develop a system**.
- ▶ Costs escalated and requirements changed.
- ▶ Agile methods intend to **develop systems more quickly with limited time spent on analysis and design**.

Agile Methods v/s Traditional Methods

- Agile methods emphasize Realtime communication, preferably face to face, over written documents.
- Agile methods like XP relies on the
 - close collaboration of actively engaged individuals with ordinary talents
 - has the ability to flexibly schedule the implementation of functionality, responding to changing business needs.

• Reference: Extreme Programming explained: Embrace Change By: Kent Beck with Cynthia Andres; 2nd ed., 2005

What is an Agile method? (1)

- ▶ **Focus on the code rather than the design.**
 - ▶ **Based on an iterative approach to software development.**
 - ▶ **Intended to deliver working software quickly.**
 - ▶ **Evolve quickly to meet changing requirements.**
-
- ▶ **There are claims that agile methods are probably best suited to small/medium-sized business systems or PC products.**

What is an agile method? (2)

- ▶ **Agile proponents believe:**
 - ▶ **Current software development processes are too heavyweight or cumbersome**
 - ▶ Too many things are done that are not directly related to software product being produced
 - ▶ **Current software development is too rigid**
 - ▶ Difficulty with incomplete or changing requirements
 - ▶ Short development cycles (Internet applications)
 - ▶ More **active customer** involvement needed

What is an agile method? (3)

- ▶ Agile methods are considered
 - ▶ **Lightweight**
 - ▶ **People-based** rather than Plan-based
- ▶ Several agile methods
 - ▶ No single agile method
 - ▶ Extreme Programming (XP) most popular
- ▶ No single definition
- ▶ Agile Manifesto closest to a definition
 - ▶ Set of principles
 - ▶ Developed by Agile Alliance

Agile Development History

- ▶ 2001 Kent Beck and 16 other software developers, referred to as “Agile Alliance”, signed the “manifesto for Agile software development” It stated We are uncovering better ways of developing software by doing it and helping others do it
- ▶ Through this work we have come to value
 - Individuals and interaction over processes and tool
 - Working software over comprehensive documentation
 - Customer collaboration over contract negotiation
 - Responding to change over following plan
- ▶ That is, while there is value in the items on the right we value the items on the left more

Agile Principles

Agile Alliance defined 12 agility principles:

1. Highest priority is to satisfy the customer through early and continues delivery of valuable software.
2. Welcome changing requirements even late n the development (for customers competitive advantage).
3. Deliver working software frequently, from couple of weeks to couple of months.
4. Business people and developers must work together daily throughout the project.
5. Build project around motivated Individuals. Give them the environment and support they need, and trust them to get the job done.
6. Most efficient and effective method of conveying information to and with team is face to face conversation.

Agile Principles

7. Working software is the primary measure of progress.

8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace.

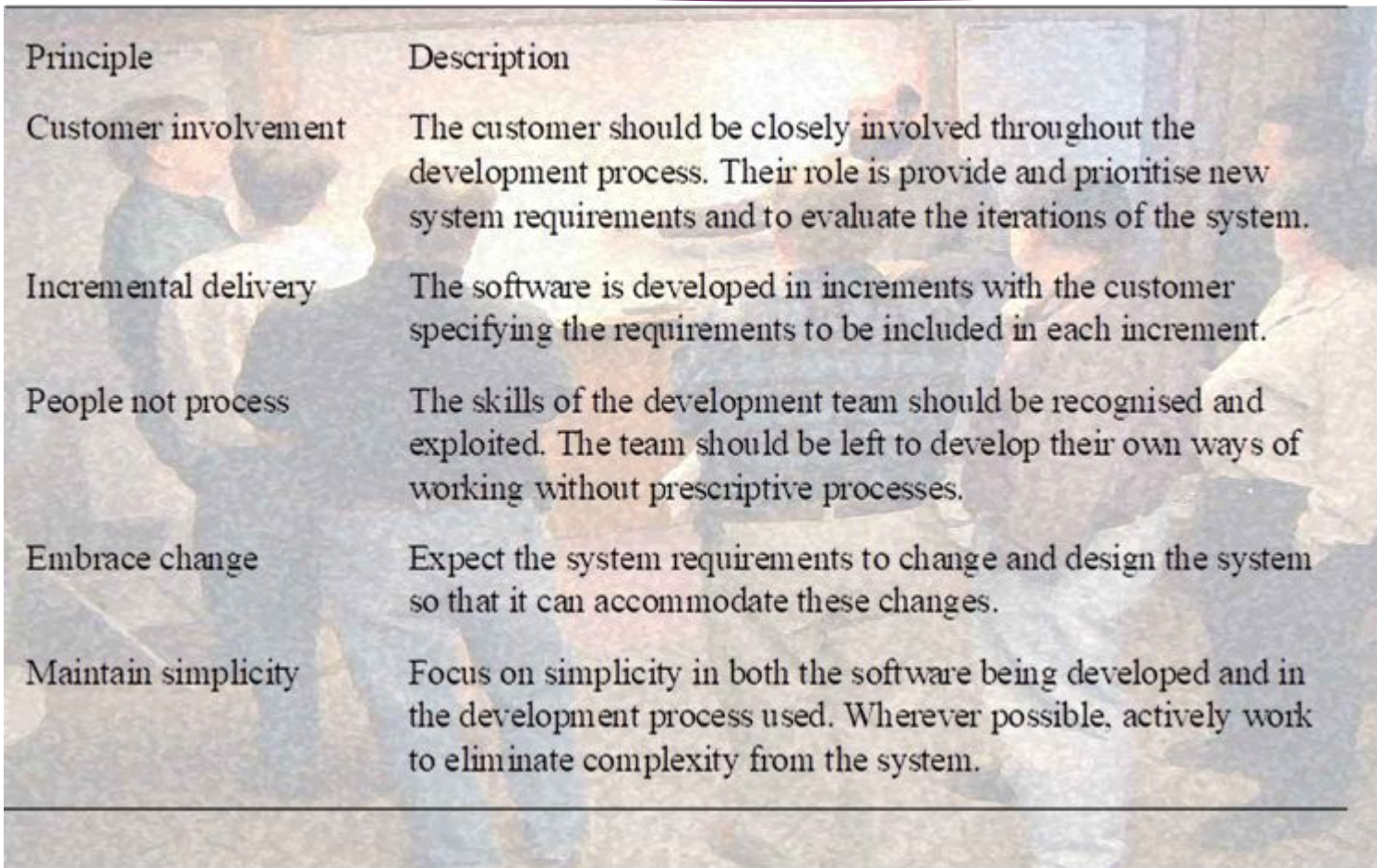
9. Continuous attention to technical excellence and good design enhances agility.

10. Simplicity the art of maximizing the amount of work done is essential.

11. The best architectures, requirements, and designs emerge from self organizing teams.

12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Summary of Principles of agile methods

A background image showing a group of people in a meeting, with some individuals looking at a screen or document. The image is slightly blurred and serves as a backdrop for the table.

Principle	Description
Customer involvement	The customer should be closely involved throughout the development process. Their role is provide and prioritise new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognised and exploited. The team should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and design the system so that it can accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process used. Wherever possible, actively work to eliminate complexity from the system.

Agile Teams.

- ▶ Hire good people;
- ▶ Seat them close together to help each other out;
- ▶ Get them close to the customers and users;
- ▶ Arrange for rapid feedback on decisions;
- ▶ Let them find fast ways to document their work;
- ▶ Cut out the bureaucracy.

Agile Teams

- ▶ Agile only works with the best developers
- ▶ Every project needs at least one experienced and competent lead person. (Critical Success Factor)
- ▶ Each experienced and competent person on the team permits the presence of 4-5 “average” or learning people.
- ▶ With that skill mix, agile techniques have been shown to work many times.

Agile variants

- ▶ Extreme Programming (XP)
- ▶ Scrum
- ▶ Adaptive Software Development (ASD)
- ▶ Crystal Clear and Other Crystal Methodologies
- ▶ Dynamic Systems Development Method (DSDM)
- ▶ Feature Driven Development
- ▶ Lean software development
- ▶ Agile Unified Process (AUP)

Extreme Programming

XP (Xtreme Programming)

- ▶ XP (according to inventor Kent Beck) is characterized by 12 practices:
 - ▶ Planning Game
 - ▶ Small Releases
 - ▶ Metaphor
 - ▶ Simple Design
 - ▶ Testing
 - ▶ Continuous Integration
 - ▶ Pair Programming
 - ▶ Collective Ownership
 - ▶ Refactoring
 - ▶ 40 Hour Week
 - ▶ On Site Customer
 - ▶ Coding Standards

Metaphor

16

- ▶ The above mentioned features of the Extreme Programming methodology formulate its 12 practices. Metaphor is one of them.
- ▶ An Extreme Programming system metaphor is a practice that is used by the XP developers to replace the standard project architecture used in traditional software development methodologies.
- ▶ **Metaphor** refers to a **simple, shared story** that helps the development team understand how the system works. A good metaphor acts as a guiding concept, making communication easier between developers and non-technical stakeholders.

Extreme programming practices (a)

Principle or practice	Description
Incremental planning	Requirements are recorded on <u>story cards</u> and the stories to be included in a release are determined by the <u>time available</u> and their relative <u>priority</u> . The developers break these stories into development 'Tasks'.
Small releases	The <u>minimal useful set of functionality</u> that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. <u>This keeps the code simple and maintainable.</u>

Extreme programming practices (b)

Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. <i>Anyone can change anything.</i>
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of <i>overtime are not considered acceptable</i> as the net effect is often to reduce code quality and medium term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.



Pair Programming (Key practice of XP)

XP (Extreme Programming)

- ▶ XP Practices
 - ▶ Collective ownership anyone can change any part of the code at any time.
 - ▶ Continuous integration new builds as soon as code ready
 - ▶ 40 hour week - maximum 40 hour week. No overtime
 - ▶ On site customer present and available full time for team
 - ▶ Coding standards rules exist and are followed
 - ▶ Open workspace large room small cubicles
 - ▶ Just rules team has own rules but can be changed any at time

Key Practices of XP

- ▶ Planning
 - ▶ User stories are written.
 - ▶ Release planning creates the schedule.
 - ▶ Make frequent small releases
 - ▶ The Project Velocity is measured.
 - ▶ The project is divided into iterations
 - ▶ Iteration planning starts each iteration.
 - ▶ Move people around
 - ▶ A stand-up meeting starts each day.
 - ▶ Fix XP when it breaks.

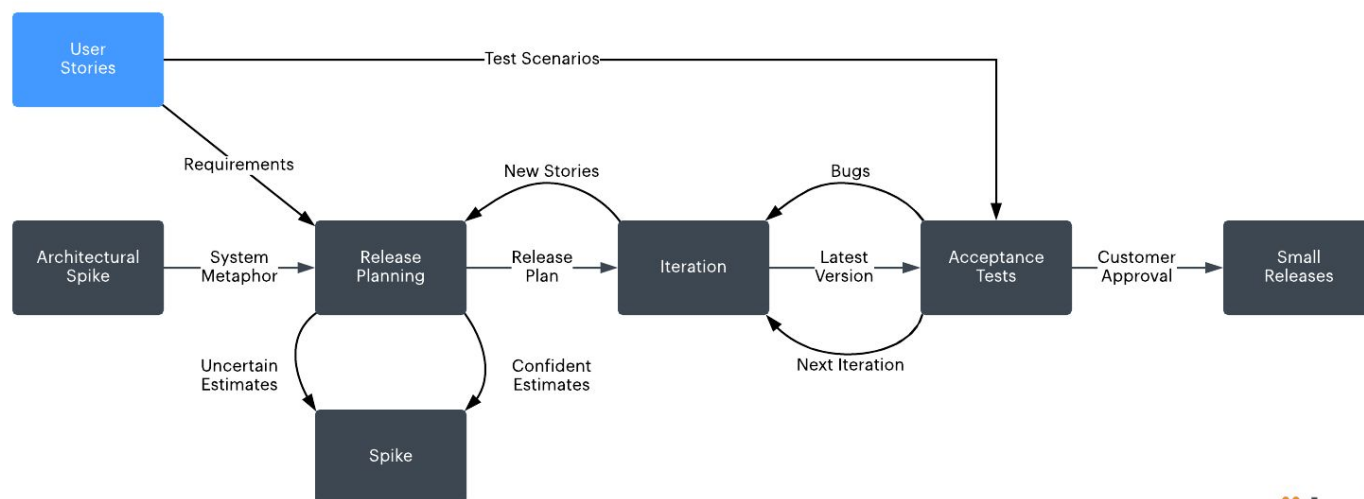
Refactoring

- ▶ Refactor whenever and wherever possible e.g.
- ▶ Techniques for improving names and location of code
- ▶ Examples
 - ▶ Move method or move field
 - ▶ move to a more appropriate class or source file
 - ▶ Rename method or rename field
 - ▶ changing the name into a new one that better reveals its purpose

Key Practices ...

- ▶ Coding
 - ▶ The customer is always available
 - ▶ Code must be written to agreed standards
 - ▶ Code the unit test first
 - ▶ All production code is pair programmed
 - ▶ Only one pair integrates code at a time
 - ▶ Integrate often
 - ▶ No overtime .
- ▶ Testing
 - ▶ All code must have unit tests
 - ▶ All code must pass all unit tests before it can be released.
 - ▶ When a bug is found tests are created.
 - ▶ Acceptance tests are run often and the score is published.

Extreme Programming (XP) Methodology



Made in
 Lucidchart

Fitting it
all
together

XP Advantages/Disadvantages

- ▶ Advantages
 - ▶ Built In Quality
 - ▶ Overall Simplicity
 - ▶ Programmer Power
 - ▶ Customer Power
 - ▶ Synergy Between Practices

XP Advantages/Disadvantages

- ▶ Disadvantages
 - ▶ Informal, little, or no documentation
 - ▶ Scalability
 - ▶ Contract Issues
 - ▶ Misconception on the cost of change
 - ▶ Tailoring

When to use / Not to use

- ▶ Applicable
 - ▶ Highly uncertain environments
 - ▶ Internal projects
 - ▶ Joint ventures
- ▶ Not Suitable / Applicable
 - ▶ Large, complex environments
 - ▶ Safety critical situations
 - ▶ Well understood requirements
 - ▶ Distant or unavailable customer

Requirements scenarios

- ✧ In XP, a **customer** or **user** is part of the XP team and is responsible for making decisions on requirements.
- ✧ User requirements are expressed as **scenarios** or **user stories**.
- ✧ These are written on cards and the development team break them down into **implementation tasks**. These tasks are the basis of schedule and cost estimates.
- ✧ The customer chooses the stories for inclusion in the next release based on their **priorities and the schedule estimates**.

A 'prescribing medication' story

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

If you select 'current medication', you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.

Examples of task cards for prescribing medication

Task 1: Change dose of prescribed drug

Task 2: Formulary selection

Task 3: Dose checking

Dose checking is a safety precaution to check that the doctor has not prescribed a dangerously small or large dose.

Using the formulary id for the generic drug name, lookup the formulary and retrieve the recommended maximum and minimum dose.

Check the prescribed dose against the minimum and maximum. If outside the range, issue an error message saying that the dose is too high or too low. If within the range, enable the 'Confirm' button.



SCRUM

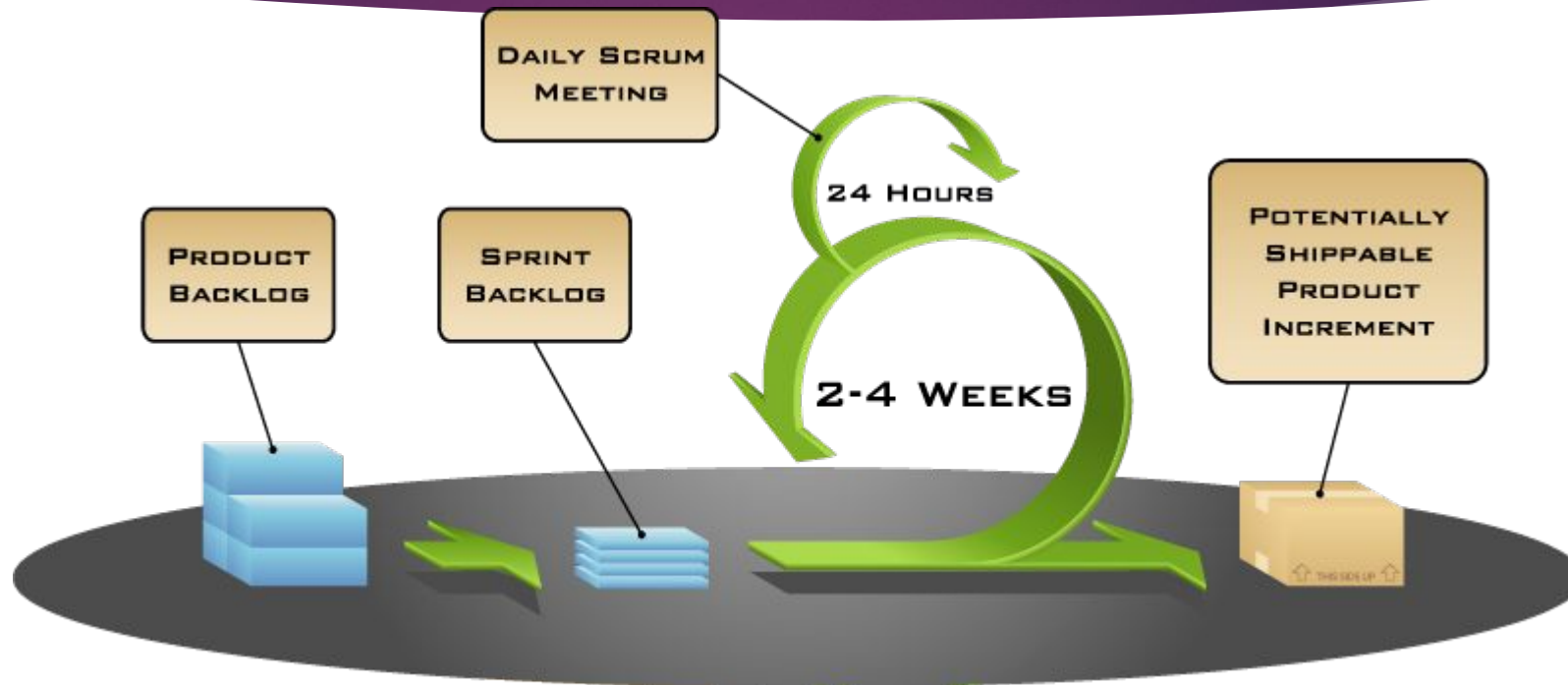
SCRUM

- ▶ First presented to OMG in 1995 by shared concerns between Advanced Development Methods (ADM) and VMARK Software (VMARK).
 - ▶ ADM produces process automation software
 - ▶ VMARK produces object-oriented software development environments
 - ▶ Both companies were concerned over the lack of breakthrough productivity being reported in object-oriented development projects
- ▶ Not an acronym, Rugby team uses SCRUM meetings
 - ▶ A scrum is a team pack in Rugby , everybody in the pack acts together with everyone else to move the ball down the field

Scrum

- ▶ The **Scrum** approach is a general agile method but its **focus is on managing iterative development** rather than specific agile practices
- ▶ There are **three phases** in Scrum:
 - ▶ The initial phase is an **outline planning** phase where you establish the general objectives for the project and design the software architecture
 - ▶ This is followed by a **series of sprint cycles**, where each cycle develops an increment of the system
 - ▶ The project **closure phase** wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project

Functionality of Scrum



Components of Scrum framework

Roles

- Product owner
- ScrumMaster
- Team

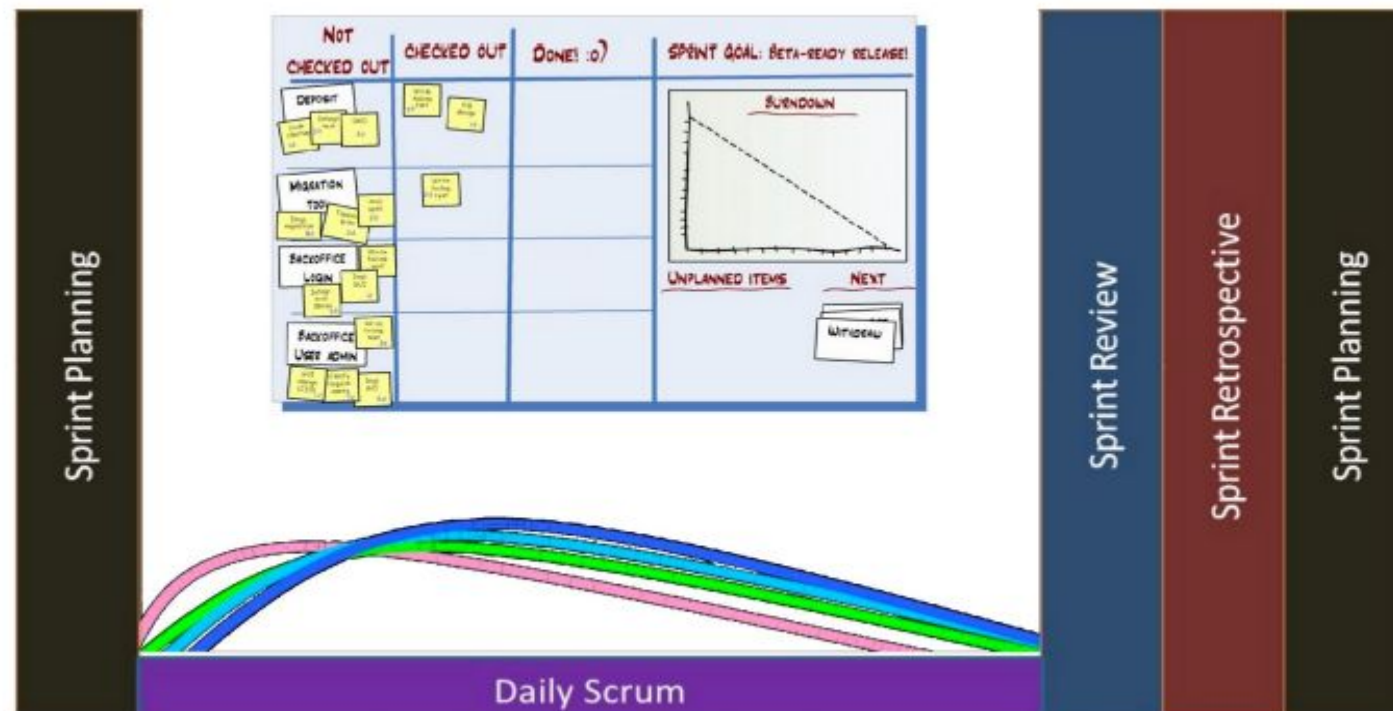
The Process

- Sprint planning
- Sprint review
- Sprint retrospective

Artifacts

- Product backlog
- Sprint backlog
- Burndown charts

Scrum Process



15 minutes maximum “stand up” meeting

What have I done?

What am I working on?

What am I stuck on/need help on?

Sprint Review:

Provide a working demo

What did we get done?

What value did we deliver?

Sprint Retrospective

What went well?

What could we have done better?

If incidents occurred: conduct a blameless postmortem

Scrum Process

Small number of team members: 6 (+/- 2)

A time-boxed model:

- Each Sprint (time box): max 30 days
- Fixed number of tasks for each Sprint
- Daily Scrum meeting: 15 min max
- Each sprint results in a
 - Sprint review (product demo): 0.5-1 hour
 - Sprint retrospective (post-mortem): 1-3 hours

The Sprint cycle

- ✧ **Sprints** are fixed length, normally 2–4 weeks. They correspond to the development of a release of the system in XP.
- ✧ The starting point for planning is the **product backlog**, which is the list of work to be done on the project
- ✧ The **selection phase** involves all of the project team who work with the customer to select the features and functionality to be developed during the sprint

Product Backlog

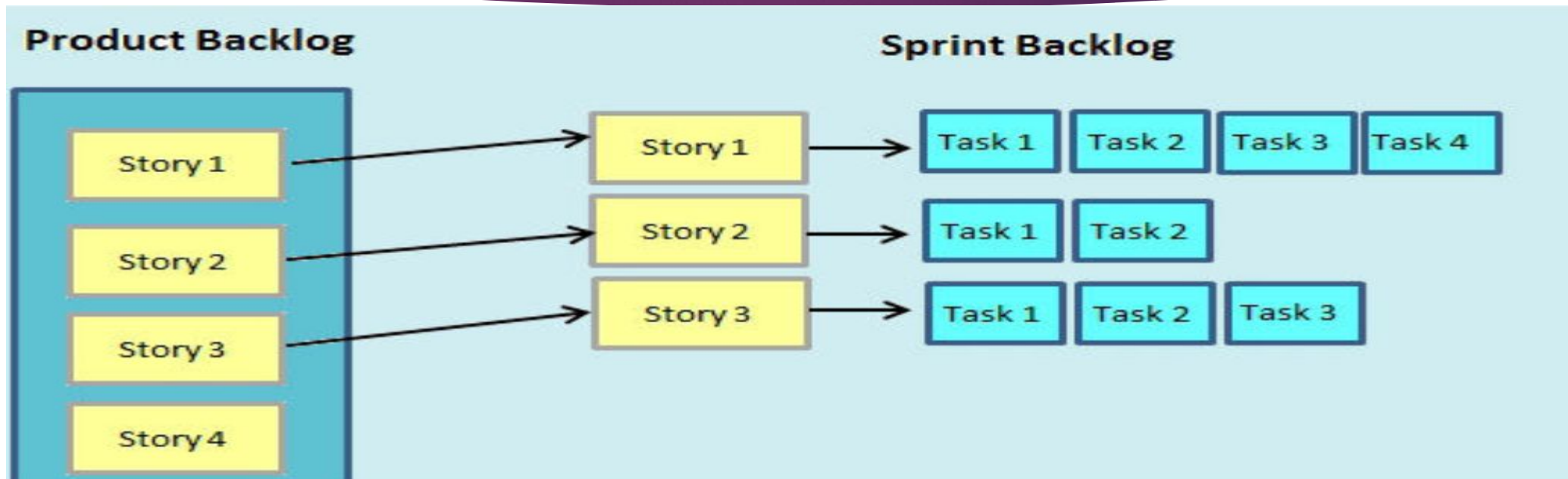
PRODUCT BACKLOG EXAMPLE

ID	As a...	I want to be able to...	So that...	Priority	Sprint	Status
1	Administrator	see a list of all members and visitors	I can monitor site visits	Must	1	Done
2	Administrator	add new categories	I can allow members to create engaging content	Must	1	Done
3	Administrator	add new security groups	security levels are appropriate	Must	1	Done
4	Administrator	add new keywords	content is easy to group and search for	Must	1	Done
5	Administrator	delete comments	offensive content is removed	Must	1	Done
6	Administrator	block entries	competitors and offenders cannot submit content	Must	1	Done
7	Administrator	change site branding	the site is future-proofed in case brand changes	Could	1	Done
8	Member	change my password	I can keep secure	Must	1	Done
9	Member	update my contact details	I can be contacted by Administrators	Must	2	Work in Progress
10	Member	update my email preferences	I'm not bombarded with junk email	Should	2	Work in Progress
11	Member	share content to social networks	I can promote what I find interesting	Could	2	Work in Progress
12	Visitor	create an account	I can benefit from member discounts	Must		To be started
13	Visitor	login	I can post new entries	Must		To be started
14	Visitor	add comments	I can have a say	Must		To be started
15	Visitor	suggest improvements	I can contribute to the site usability	Should		To be started
16	Visitor	contact the Administrators	I can directly submit a query	Could		To be started
17	Visitor	follow a member's updates	I'm informed of updates from members I find interesting	Should		To be started
18	Visitor	view a member's profile	I can know more about a member	Must		To be started
19	Administrator	generate incoming traffic report	I can understand where traffic is coming from	Must		To be started

The Sprint cycle

- ✧ Once these are agreed, the team organize themselves to develop the software. During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called **Scrum master**
- ✧ The role of the Scrum master is to protect the development team from external distractions
- ✧ At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Sprint Backlog



- ✧ No more than 300 tasks in the list
- ✧ If a task requires more than 16 hours, it should be broken down
- ✧ Team can add or subtract items from the list. Product Owner is not allowed to do it

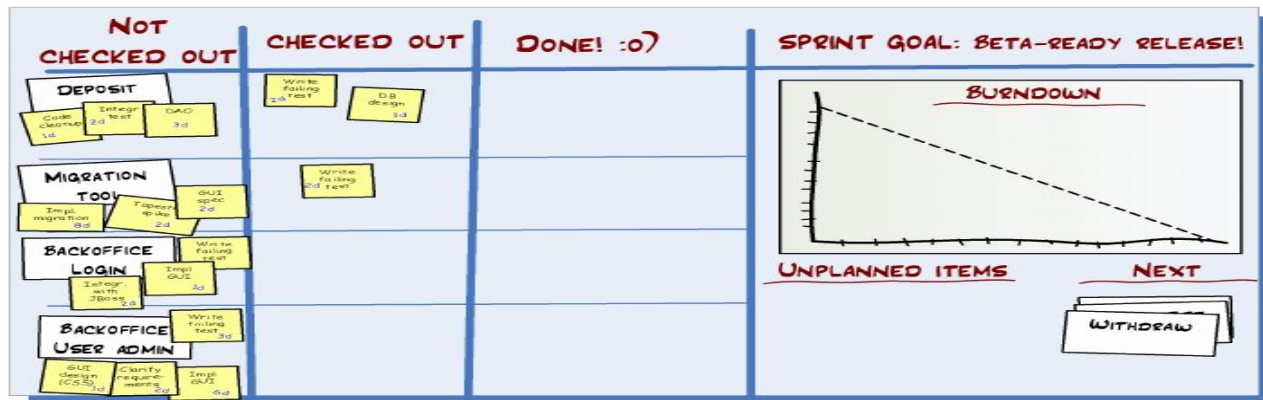
Sprint Backlog -Example

SPRINT [NUMBER] BACKLOG

ID	User Story	Tasks	Owner	Status	Estimated effort	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6
US001	As a call centre agent I need to be able to see the caller's previous tickets in their contact record.	Install standard contact record module	J Smith	Completed	16	0.5	0	0	0	0	0
		Demo standard module and identify revised configuration.	J Smith	Completed		0	8	0	0	0	0
		Complete custom configuration as agreed in the demo.	J Smith	Completed		0	0	6	2	0	0
US002	As a customer I need to be able to login to my account from any page on the website.	Install login widget on website.	F Dole	Completed	48	0	0	5	7	2	8
		Apply customer branding.	J Smith	Completed		0	0	0	0	0	0
US003	As a customer I need to be able to look up my address using my postcode.	Install contact page.	P Murphy	Completed	24	0	0	0	0	0	0
		Integrate postcode plugin to contact page.	P Murphy	Completed		0	0	0	0	0	0
		Demo plugin to customer	J Smith	Completed		0	0	0	0		



Daily Scrum



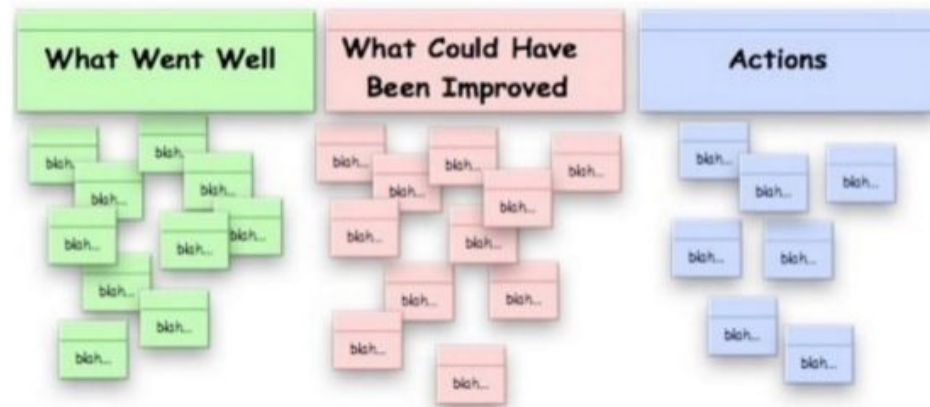
Max. 15 minutes

What did you do yesterday?

What will you do today?

Is there anything in your way?

Scrum: sprint retrospective



Stop doing



Continue doing



Start doing

Velocity

- ✧ **Velocity** is a measure of the amount of work a Team can tackle during a single Sprint and is the key metric in **Scrum**. **Velocity** is calculated at the end of the Sprint by totaling the Points for all fully completed User Stories.
- ✧ A **burn down chart** is a graphical representation of work left to do versus time. The outstanding work (or backlog) is often on the vertical axis, with time along the horizontal. That is, it is a run **chart** of outstanding work. It is useful for predicting when all of the work will be completed.

Sprint - Duration

- ✧ Sprint Duration – 2 weeks
- Team Size - 7
- Hours/Day – 6
- Total Capacity – 420 hours

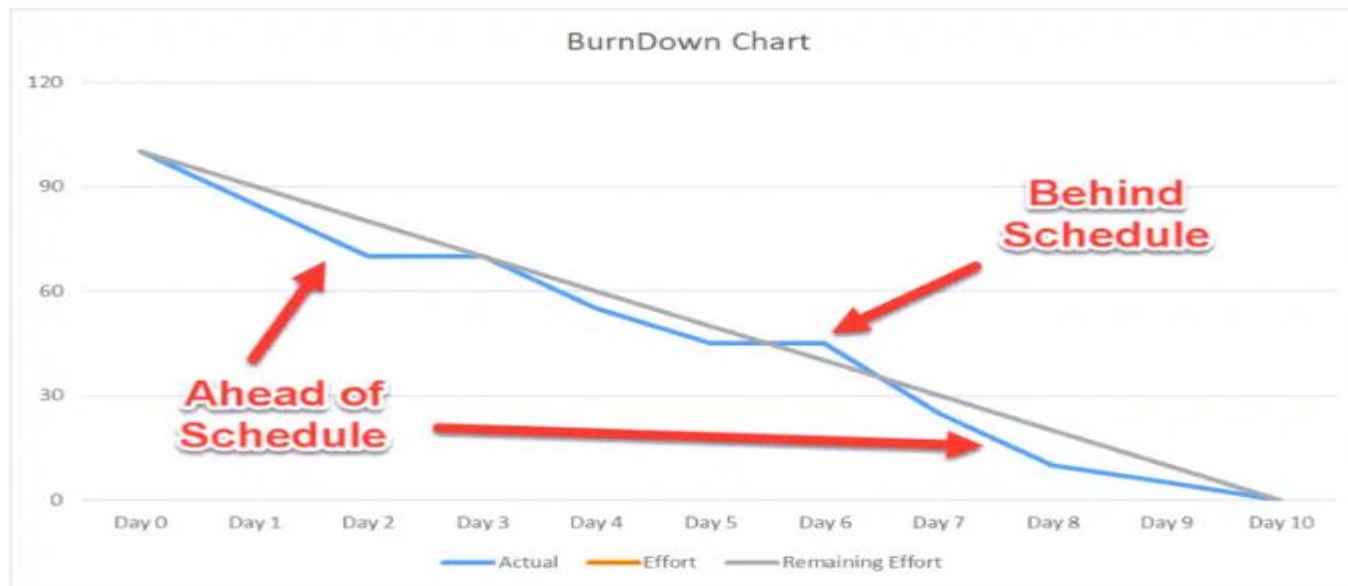


TABLE 1: KEY SCRUM CONCEPTS

Concept	Description
Project	Discreet set of end user requirements that have been grouped, prioritised and funded.
Requirement	The end user statement that outlines their information need.
Sprint	A sprint is a 1 to 4 week time-boxed event focused on the delivery of a subset of User Stories taken from the Project Backlog.
Project Backlog	The Project Backlog is the current list of User Stories for the Project. User Stories can be added, modified or removed from the Backlog during the Project.
Sprint Backlog	Subset of User Stories from the Project Backlog that are planned to be delivered as part of a Sprint.
User Stories	The User Story is a one or two line description of the business need, usually described in terms of features.
Tasks	Tasks are the activities performed to deliver a User Story.
Technical Debt	This refers to items that were either: • missing from the Planning meeting; or • deferred in favor of early delivery.

User Story Examples

- ▶ Standard template of a user story :
- ▶ As a < type of user>
- ▶ I want < some goal>
- ▶ so that < some reason>
- ▶ Ex1: As a power user, I can specify files or folders to backup based on file size, date created and date modified.
- ▶ Ex2: As a user, I can indicate folders not to backup so that my backup drive isn't filled up with things I don't need saved.



How is detail added to user stories?

- ▶ Detail can be added to user stories in two ways:
 - ▶ By splitting a user story into multiple, smaller user stories.
 - ▶ By adding “conditions of satisfaction.”
- ▶ Ex1: As a vice president of marketing, I want to select a holiday season to be used when reviewing the performance of past advertising campaigns so that I can identify profitable ones.
- ▶ Detail could be added as:
 - ▶ Make sure it works with major retail holidays: Christmas, Easter, President's Day, Mother's Day, Father's Day.
 - ▶ Holiday seasons can be set from one holiday to the next (such as Thanksgiving to Christmas).
 - ▶ Holiday seasons can be set to be a number of days prior to the holiday.



Story ID:

Story Title:

User Story:

As a: <role>

I want: <some goal>

So that: <some reason>

Importance:

Estimate:

Acceptance Criteria

And I know I am done when:

Type:

- ☐ Search
- ☐ Workflow
- ☐ Manage Data
- ☐ Payment
- ☐ Report/ View

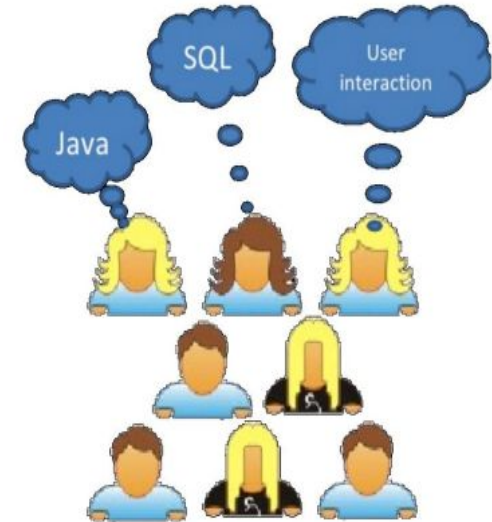
THREE MAIN SCRUM ROLES



Product owner
(*Customer*)



Scrum master
(*Manager/Moderator*)



Scrum team
(*Tech experts*)

The ScrumMaster

- ▶ Represents management to the project
- ▶ Responsible for enacting Scrum values and practices
- ▶ Removes impediments
- ▶ Ensure that the team is fully functional and productive
- ▶ Enable close cooperation across all roles and functions
- ▶ Shield the team from external interferences

Product Owner

- ▶ Define the features of the product
- ▶ Decide on release date and content
- ▶ Be responsible for the profitability of the product (ROI)
- ▶ Prioritize features according to market value
- ▶ Adjust features and priority every iteration, as needed
- ▶ Accept or reject work results

The team

- ▶ Typically 5-9 people
- ▶ Cross functional:
 - ▶ Programmers, testers, user experience designers, etc.
- ▶ Members should be full time
- ▶ May be exceptions (e.g., database administrator)
- ▶ Teams are self organizing
 - ▶ Ideally, no titles but rarely a possibility
- ▶ Membership should change only between sprints

